

Файлове и файлови системи — подробен конспект

1. Файлове и техните основни атрибути

1.1. Какво е файл и защо съществува

На физическо ниво дискът е просто **огромен масив от блокове** (например 512 байта или 4 KB всеки), номерирани от 0 до N. Ако работим директно с тях, ще трябва да помним: „моят документ е в блокове 1024, 1025 и 2048“. Това е непоносимо.

Файлът е абстракция, която операционната система (ОС) предлага, за да:

- дадем **име** на колекция от свързани байтове;
- работим с тях като с **непрекъсната последователност**, без значение къде физически са разпръснати по диска;
- имаме **операции** върху тях (отвори, чети, пиши, премести курсора), а не върху блокове.

Интуицията: файлът е като **книга в библиотека**. Не ни интересува на кой рафт точно са страниците — даваме името и библиотекарят (ОС) ни ги донася в правилния ред.

1.2. Основни атрибути на файла (метаданни)

Метаданните са „информация за информацията“ — не самото съдържание, а данни за него. В UNIX/Linux те се пазят в структура, наречена **inode** (index node):

Атрибут	Какво описва	Защо е нужен
Име	низ за човешка идентификация	имената живеят в директории, не в inode
Размер	колко байта има файлът	за read/seek, за визуализация
Тип	обикновен, директория, устройство, символен линк, pipe, socket	ОС знае как да го интерпретира
Собственик (UID) и група (GID)	кой го притежава	основа на защитата
Права за достъп (mode)	rwx за собственик/група/всички	контрол на достъпа
Времеви маркери	atime (последен достъп), mtime (последна модификация), ctime (последна промяна на inode)	за backup, за инструменти като <code>make</code>
Брой твърди връзки (link count)	колко имена сочат към този inode	когато стане 0, файлът се изтрива
Указатели към блокове с данни	къде физически са байтовете	сърцето на inode
Брой блокове	физически заети блокове на диска	може да е различно от размера (sparse files)

Ключова идея: името НЕ е в inode. Името е в директорията. Това обяснява защо един файл може да има няколко имена (твърди връзки — hard links) и защо преименуването е евтино — променяме само записа в директорията.

2. Файловата система като абстракция за организация на обекти

2.1. Единно пространство от именувани обекти

Структура

Във Unix-подобните ОС всички обекти живеят в **едно общо дърво**, чиято връзка е `/` (root). За разлика от Windows (където имаме `C:\`, `D:\` и т.н.), тук **няма отделни „устройства“ с букви** — всичко е едно дърво.

```
/
├─ bin/      изпълними програми
├─ etc/      конфигурации
├─ home/
│   ├─ ivan/
│   └─ maria/
├─ usr/
├─ var/
├─ dev/      специални файлове за устройства
├─ proc/     виртуална ФС с информация за процесите
└─ mnt/      обичайно място за монтиране
```

Защо единно дърво? Защото така **всеки обект има уникален път** (path). Пътят е низ от имена, разделени с `/`. От него точно се вижда къде в йерархията е обектът:

```
/home/ivan/documents/cv.pdf
```

Това е семантично различно от Windows подхода — там програма, която иска да отвори файл, трябва първо да знае на кое устройство е. В Unix просто получава път и не я интересува дали данните идват от SSD, мрежов диск или USB.

Физическо представяне

Едно дърво **не означава един диск**. Реалността е, че:

- кореновата ФС обикновено е на главния SSD/HDD;
- `/home` може да е на втори диск;
- `/mnt/usb` може да е USB флашка;

- `/proc` въобще не е на диск — генерира се динамично от ядрото.

Това магически работи чрез **точки на монтиране**.

Точки на монтиране (mount points)

Монтирането е операция, която „закача“ дървото на една ФС в произволна **директория** на вече съществуващото дърво.

Пример: имаме USB с файлова система. Командата:

```
mount /dev/sdb1 /mnt/usb
```

казва на ядрото: „когато някой обърне до `/mnt/usb`, не гледай в старата директория, а отиди в ФС, която е на `/dev/sdb1`“.

Интуиция: мисли си за това като да **сложиш дърво в дупка** на друго дърво. Това, което е било в `/mnt/usb` преди, става недостъпно (но не се изтрива — появява се пак след `umount`).

Защо това е мощно?

- Можем да преместим даден поддиректориум на друг, по-бърз диск, без приложенията да забележат.
- Можем да монтираме мрежови ФС (NFS, SMB) и да изглеждат като локални.
- `/proc`, `/sys` са „виртуални ФС“ — съдържанието им е fabrication, генерирано от ядрото при четене.

2.2. Видове обекти

Във Unix казваме „всичко е файл“, което не значи буквално текст или документ. Имаме различни **видове** обекти, които ползват едни и същи функции

(`open/read/write/close`):

Обикновени файлове (regular files)

Поредица от байтове на диска. Може да са текст, изображение, изпълнима програма — за ФС те са просто байтове.

Директории

Директорията е специален файл, чието съдържание е таблица от записи (`име, inode_номер`). Когато правим `ls`, ОС всъщност чете тази таблица.

Това обяснява:

- защо имената на файлове в Unix не са в самия файл, а в директорията, която го съдържа;
- защо един inode може да има няколко имена в различни директории — това са твърди връзки;
- защо `mv` в рамките на едно ФС е почти безплатно — само се променя запис в директория.

Устройства (device files)

Файлове в `/dev`, които **не съдържат данни**, а представляват хардуерно устройство. Делят се на:

- **Блокови (block devices)** — `/dev/sda`, `/dev/sdb1`. Достъп на блокове с произволен ред (random access). Например дискове.
- **Символни (character devices)** — `/dev/tty`, `/dev/null`. Достъп по байтове, обикновено последователно. Например клавиатура, серийни портове.

Когато пишеш в `/dev/sda` пишеш направо на диска (нужни са root права!). Това е силата на абстракцията: **програма за копиране може да копира файл-към-файл или файл-към-устройство със същия код.**

Информационни канали (pipes, FIFOs)

Канал между процеси. Когато пишеш `ls | grep txt`, shell-ът създава анонимна тръба:

- единият край е свързан с stdout на `ls`;
- другият край — със stdin на `grep`.

Има и **именувани тръби (FIFO)**, които живеят като файл в ФС:

```
mkfifo /tmp/my_pipe
```

Процес А пише в `/tmp/my_pipe`, процес В чете оттам — без да си знаят PID-овете.

Интуицията: тръбата не пази данни на диск. Тя е буфер в паметта. Когато всички, които пишат, я затворят, четящият получава EOF.

Символни връзки (symbolic links / soft links)

Файл, чието съдържание е **път** към друг файл. За разлика от твърдата връзка, може да сочи извън ФС и към директория, и може да бъде „счупен“ (target file не съществува).

2.3. Основни функции на файловата система

а) Достъп до информационните обекти

ФС превежда от името (път) към физическата позиция:

```
/home/ivan/cv.pdf → inode 73221 → блокове 14512, 14513, 81223
```

Това става на стъпки (path resolution): прочети `/`, намери `home`, отиди в inode-а на `home`, намери `ivan`, и т.н.

б) Изоляция и защита между потребителите

Системата има потребители (UID) и групи (GID). Всеки процес има идентичност, наследена от потребителя, който го е стартирал. Ядрото проверява при всяка операция дали процесът има право да я извърши върху съответния обект.

в) Управление на правата за достъп

Класическият Unix модел (POSIX) дефинира **три права** (read, write, execute) за **три категории** (user/owner, group, others). Това дава 9 бита, представяни като:

```
rwXr-x--- = 750 в осмично
```

- `user` (собственик): чете, пише, изпълнява;
- `group`: чете и изпълнява;
- `others`: нищо.

За директория значенията се променят:

- `r` — можеш да видиш списъка с имена;
- `w` — можеш да създаваш/изтриваш файлове в нея;
- `x` — можеш да „влизаш“ в нея и да обръщаш до файловете ѝ (без `x` не можеш дори да отвориш файл, дори да знаеш името).

Има и специални битове (setuid, setgid, sticky), и съвременни разширения — **POSIX ACL** (по-фини права за конкретни потребители).

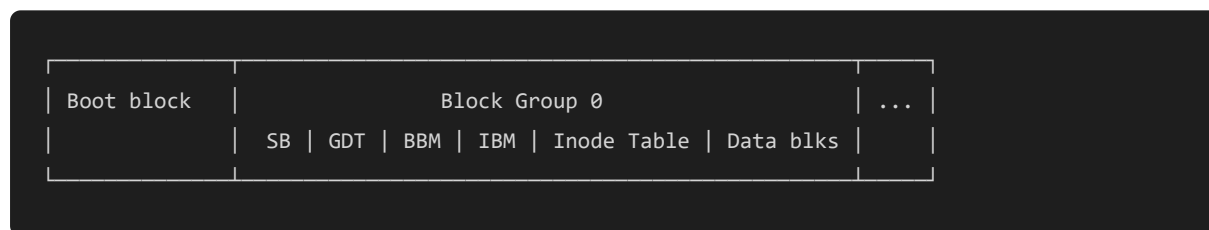
3. Реализация на конкретна файлова система

3.1. Стандарти за представяне — ext2/ext3 в детайли

ext2 (Second Extended Filesystem) е класическата Linux ФС от 90-те. **ext3** е същата, но с добавено **журналиране** (виж по-долу). Структурата е почти идентична.

Подразделение на дяла на блокови групи

Цялото устройство е разделено на блокове (обикновено 4 KB). Тези блокове са организирани в **block groups** — групи от блокове, всяка със своя структура:



Какво има във всяка група:

- **Superblock (SB)** — глобална информация: размер на блок, общ брой inode-и, брой свободни блокове, magic number за разпознаване. Дублира се в няколко групи (за надеждност).
- **Group Descriptor Table (GDT)** — описва всички групи.
- **Block bitmap (BBM)** — битова карта на свободните блокове в групата (1 бит на блок).
- **Inode bitmap (IBM)** — битова карта на свободните inode-и.
- **Inode table** — фиксиран брой inode-и за тази група.
- **Data blocks** — реалните данни на файловете.

Защо групи, а не една плоска структура? Защото има смисъл inode-ите и данните на даден файл да са физически близо — диска по-малко „търси“. При разпределяне ФС се опитва inode и блокове да са в една и съща група.

Inode структура и адресиране на блокове

Един inode съдържа около 12 директни указателя към блокове + индиректни:

- **12 директни указателя** — за малки файлове са достатъчни ($12 \times 4 \text{ KB} = 48 \text{ KB}$).
- **1 единичен индиректен** — указател към блок, който съдържа указатели към блокове с данни. Ако в блок се събират 1024 указателя, това е още 4 MB.
- **1 двоен индиректен** — указател към блок с указатели към блокове с указатели. Около 4 GB.
- **1 троен индиректен** — за огромни файлове.

```
inode
├─ direct[0..11] → data block
├─ single_indirect → [ptr, ptr, ...] → data blocks
├─ double_indirect → [ptr, ...] → [ptr, ...] → data blocks
└─ triple_indirect → ...
```

Интуицията: това е като оглавление на книга, в което повечето глави са изброени директно, но за дебели книги има оглавление-на-оглавления. Малките файлове са бързи (един преход), големите — по-бавни, но достъпни.

3.2. Ускоряване на достъпа

Дисковете са с порядъци по-бавни от RAM (милисекунди срещу наносекунди). Всичко в дизайна на ФС се върти около това да се минимизират достъпите до диск.

Кеширане на вход/изхода (buffer cache, page cache)

ОС държи в RAM **копие на често използвани блокове**. Когато процес чете блок, който е в кеша — връща се директно от RAM, без преход до диска.

Стратегии за изселване (eviction): LRU (least recently used) — изхвърля се блокът, до който отдавна не е имало достъп. Кешът се пълни автоматично и заема свободна RAM.

Това е причината вторият път да четеш голям файл да бъде мигновено — той вече е в кеша.

Отлагане на записа (write-back caching)

Когато пишем в файл, ОС **не записва веднага на диска**. Вместо това:

1. Променя блока в RAM кеша и го маркира като „мръсен“ (dirty).
2. Връща управление на програмата веднага.
3. По-късно (след секунди или при натиск от паметта), блокът се записва физически.

Защо: ако пишем 100 пъти по 1 байт в един и същи блок, имаме само 1 физически запис. Освен това, могат да се обединяват и пренареждат записи.

Цената: ако токът отиде преди flush, променените данни се губят. Точно за това има `fsync()` (форсира flush) и **журналиране** (виж по-долу).

Алгоритъм на асансьора (elevator algorithm, SCAN/LOOK)

Когато имаме много заявки за четене/запис на разпръснати блокове, **редът на изпълнение има значение**, защото механичните дискове губят време при движение на главата.

Алгоритъмът на асансьора: главата се движи в **една посока** и обслужва всички заявки по пътя си, после се връща в обратната посока. Точно като асансьор в сграда — не отива на 1-ия етаж, после на 7-я, после на 2-я; отива по ред нагоре, после надолу.

```
Заявки за блокове: 4, 27, 9, 15, 2, 32
Класически (FIFO): 4 → 27 → 9 → 15 → 2 → 32 (много туп-туп)
Алгоритъм на асансьора: 2 → 4 → 9 → 15 → 27 → 32 (плавно)
```

В съвременните SSD-та няма движещи се части, така че този алгоритъм има по-малко значение, но се запазват други оптимизации (групиране, паралелизъм).

3.3. Надеждна реализация

Журнална файлова система (journaling)

Проблемът: при отлагане на записа, **една операция (например преименуване) може да включва няколко промени в различни структури** (директория, inode, bitmap). Ако токът прекъсне между тях, ФС е в **непоследователно състояние** — например изтрит файл, чийто inode не е освободен.

Решение: **журналът** е специален участък в ФС, в който преди да се направят реалните промени, се записва „**какво ще се направи**“ (transaction record). Стъпки при писане:

1. Запиши в журнала: „ще променя блок 1024, inode 5, bitmap за блок 1024“.
2. Чакай потвърждение, че журналът е на диска.
3. Запиши в журнала „commit“ маркер.
4. Извърши реалните промени.
5. Когато всичко е готово, изтрий от журнала.

Какво става при срыв:

- ако сме спрели преди commit — всичко в журнала се отхвърля, ФС е в старо състояние;
- ако сме спрели след commit — при ребоот, ФС изпълнява операциите от журнала (replay).

И в двата случая ФС е **консистентна**.

ext3 добавя точно такъв журнал върху ext2. Има три режима:

- **journal** — журналират се и метаданни, и данни (най-сигурно, най-бавно);
- **ordered** — журналират се само метаданни, но първо се записват данните и после метаданните (по подразбиране);
- **writeback** — журналират се само метаданни, без подреденост (бързо, но при срыв може да четеш стари данни от нов файл).

Аналогия: журналът е като **чернова на пощальон**, която казва „ще доставя 5 писма“. Дори ако той падне на път, заместник може да види черновата и да довърши, или да види че е празна — и нищо не е почнато.

RAID — Redundant Array of Independent Disks

Дори с журнал, ако дискът физически се счупи, данните се губят. RAID комбинира **няколко диска**, така че:

- да устои на повреда на 1 или повече дискове (надеждност);
- да ускори (паралелен достъп);
- или и двете.

RAID 1 — Mirroring (огледало)

Двата (или повече) диска съдържат **идентични копия**. Всяка операция за запис се изпълнява на всички. Четене може да се прави от всеки.

```
Disk A: [Блок1] [Блок2] [Блок3] [Блок4]
Disk B: [Блок1] [Блок2] [Блок3] [Блок4] (идентичен)
```

Плюсове:

- Издържа на загубата на 1 диск (от 2);
- Четенето може да е по-бързо (паралелизъм);

- Възстановяването е тривиално — копиране от оцелелия диск.

Минуси:

- Използваме само 50% от капацитета (плащаме два диска, ползваме един);
- Записът не е по-бърз (трябва да се напише на всички дискове).

Подходящо за: критични малки масиви, бази данни, OS дискове.

RAID 5 — Striping with Distributed Parity

Използват се **поне 3 диска**. Данните се „нарязват“ на ленти (stripes) и се разпределят. За всеки stripe се пресмята **parity** (XOR на данните), който се съхранява на различен диск за различните stripes (затова е distributed).

	Disk 1	Disk 2	Disk 3
Stripe 1:	A1	A2	$P(A1 \oplus A2)$
Stripe 2:	B1	$P(B1 \oplus B2)$	B2
Stripe 3:	$P(C1 \oplus C2)$	C1	C2

Ако един диск откаже, можем да възстановим данните му чрез XOR на останалите.

(Свойство: ако $A \oplus B = P$, то $A = B \oplus P$ и $B = A \oplus P$.)

Плюсове:

- Издържа на загубата на 1 диск (от N);
- Полезен капацитет = $(N-1)/N$ (например 3 диска → 67% полезни, много по-добре от RAID 1);
- Четенето е бързо (паралелизъм между дискове).

Минуси:

- Записът е по-бавен — изисква четене на старите данни и стария parity, пресмятане на нов parity и два записа („RAID 5 write penalty“).
- При повреда на 1 диск, до възстановяването системата е уязвима — ако и втори диск откаже, всички данни се губят.
- Възстановяването е бавно при големи дискове (часове до дни).

Подходящо за: големи масиви със смесено натоварване, файлови сървъри.

Интуицията за XOR parity: ако имаш числата 5, 3, 6 и записваш сумата 14, после загубиш 3-ката, можеш да я възстановиш: $14 - 5 - 6 = 3$. XOR работи по същия начин — обратима, но за битове.

4. POSIX функции за работа с файлове

POSIX (Portable Operating System Interface) дефинира **стандартен интерфейс** за работа с файлове, така че кодът да е преносим между Unix-системите.

Ключово понятие: **file descriptor (fd)**. Това е малко цяло число (0, 1, 2, ...), което ОС връща при отваряне на файл. Чрез него по-нататък указваме на коя „отворена сесия“ работим. Всеки процес има своя таблица на fd-та; ядрото пази състоянието (позиция на курсора, режим на отваряне).

По подразбиране всеки процес има:

- fd 0 = stdin (стандартен вход)
- fd 1 = stdout (стандартен изход)
- fd 2 = stderr (стандартна грешка)

4.1. `open()` — отвори файл

```
int open(const char *pathname, int flags, mode_t mode);
```

Какво прави: превръща име на файл (`pathname`) във file descriptor. ОС намира inode-a, проверява правата, създава запис в таблицата на отворените файлове на процеса.

Параметри:

- `pathname` — пътят до файла (абсолютен `/home/ivan/x.txt` или относителен `./x.txt`).
- `flags` — режим, комбинирани с `|`:
 - `O_RDONLY` — само за четене;
 - `O_WRONLY` — само за писане;
 - `O_RDWR` — четене и писане;

- `O_CREAT` — създай ако не съществува (тогава трябва и `mode`);
- `O_EXCL` — заедно с `O_CREAT` гарантира, че файлът е нов (иначе грешка);
- `O_TRUNC` — изпразни го при отваряне;
- `O_APPEND` — пиши винаги в края.
- `mode` — права при създаване (например `0644`).

Връща: `fd` ≥ 0 при успех, `-1` при грешка (и `errno` се задава: `EACCES`, `ENOENT` и т.н.).

```
int fd = open("/tmp/log.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
if (fd < 0) { perror("open"); exit(1); }
```

Интуиция: като да поискаш ключ за стая от рецепцията. Получаваш номерче (`fd`), което носиш при всяка следваща заявка.

4.2. `close()` — затвори файл

```
int close(int fd);
```

Какво прави: освобождава `fd`-то. ОС маркира файла като затворен, изсипва (`flush`) евентуални буфери, освобождава ресурси.

Защо е важно да затваряме: всеки процес има лимит на отворените `fd`-та (типично 1024). Ако забравим `close()`, имаме „file descriptor leak“, който в дълготрайни процеси (сървъри) води до срыв.

```
close(fd);
```

4.3. `read()` — чети от файл

```
ssize_t read(int fd, void *buf, size_t count);
```

Какво прави: копира до `count` байта от файла (от текущата позиция) в буфера `buf`. Премества позицията напред с прочетените байтове.

Важно: може да прочете **по-малко** от `count` байта, дори при нормална работа:

- ако е достигнат край на файла;
- ако е тръба/socket и наличните данни са по-малко;
- ако сигнал е прекъснал четенето.

Връща:

- `> 0` — брой прочетени байтове;
- `0` — край на файла (EOF);
- `-1` — грешка.

```
char buf[4096];
ssize_t n = read(fd, buf, sizeof(buf));
if (n < 0) perror("read");
else if (n == 0) printf("EOF\n");
else fwrite(buf, 1, n, stdout);
```

Чест бъг: да предположиш, че винаги ще се прочетат `count` байта. Правилно — в цикъл, докато не върне 0.

4.4. `write()` — пиши във файл

```
ssize_t write(int fd, const void *buf, size_t count);
```

Какво прави: копира `count` байта от `buf` във файла (на текущата позиция). При `O_APPEND` винаги в края, без значение от позицията.

Важно: както при `read`, може да запише по-малко от поисканото — особено за тръби и сокети. Обикновените файлове на диск обикновено пишат всичко, но коректният код проверява.

Връща: брой записани байтове или -1.

```
const char *msg = "Hello\n";
ssize_t n = write(fd, msg, strlen(msg));
if (n < 0) perror("write");
```

Внимание: `write` връща след като данните са в **кеша на ОС**, не на диска! За да гарантираме физически запис: `fsync(fd)`.

4.5. `lseek()` — премести позицията на курсора

```
off_t lseek(int fd, off_t offset, int whence);
```

Какво прави: премества „курсора“ (file position indicator) във файла. Следващото четене/писане ще започне от новата позиция.

Параметри:

- `offset` — отместване (може да е отрицателно);
- `whence` — спрямо коя точка:
 - `SEEK_SET` — от началото на файла (`offset` = абсолютна позиция);
 - `SEEK_CUR` — от текущата позиция (relative);
 - `SEEK_END` — от края на файла.

Връща: новата позиция или -1.

```
lseek(fd, 0, SEEK_SET);           // в началото
lseek(fd, 0, SEEK_END);           // в края (например за добавяне)
off_t pos = lseek(fd, 0, SEEK_CUR); // вземи текуща позиция
lseek(fd, 100, SEEK_SET);          // прескочи на байт 100
```

Интересно: `lseek` може да направи позицията **след края на файла**. Ако след това запишеш, се получава **sparse file** — между стария край и новата позиция има „дупка“ от нули, която не заема място на диска (наистина се записват само реалните блокове).

Защо това е полезно: бази данни, торент файлове (резервираш голям файл, който се пълни на части), виртуални дискове.

5. Обобщителна интуиция

Запомни три нива на абстракция:

1. **Физическо ниво** — блокове, сектори, дискове, RAID.
2. **Логическо ниво (ФС)** — inode-и, директории, журнал, кеш.
3. **Програмен интерфейс (POSIX)** — open/read/write/close/lseek с file descriptors.

Всяко по-високо ниво **скрива сложността** на по-долното. Това позволява един и същи код:

- да чете от локален SSD, мрежов диск или USB;
- да пише в реален файл, тръба или принтер;
- да работи коректно дори при изключване на тока (благодарение на журнала);
- да преживее повреда на диск (благодарение на RAID).

Цялата дисциплина „операционни системи“ е една дълга история за това как с правилните абстракции **сложното става просто** за програмиста, без да губим производителност и надеждност.